

Basic Computer Skills for Digital Forensics

Idriss Muhammad Abdullahi

2025/FWSD/11475

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

04/August/2026

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

A practical foundation for interpreting digital evidence at byte, text, time and memory levels

Lab Type	Guided individual practical with report submission
Estimated Duration	2-3 hours
Recommended Platform	Kali Linux, Ubuntu Linux, WSL, or any Linux VM with Python 3 installed
Submission Format	One PDF report containing answers, screenshots, commands used, explanations and reflection
Academic Integrity	Each student must generate their own outputs and screenshots. Copied screenshots or reports should not be accepted.

Mandatory preparation before starting

Students must first go through the course slide material titled "Understand Number Systems and Timestamps". The slide is the walkthrough for this lab. Do not proceed to the practical tasks until you understand the concepts of decimal, binary, hexadecimal, ASCII, epoch time, hashing and endianness.

1. Lab Overview

Scenario: You are a beginner digital forensics trainee. During an investigation, you receive values from disk sectors, command-line outputs, logs, hashes and memory dumps. Your task is to correctly interpret numbers, text encodings, timestamps and byte order so that evidence is not misunderstood.

1.1 Learning Outcomes

Differentiate decimal, binary and hexadecimal number systems and explain why they matter in digital forensics.

Convert values manually and validate conversions using Linux shell arithmetic, bc and Python.

Interpret ASCII and UTF-8 text from hexadecimal evidence strings.

Convert Unix epoch timestamps and understand other forensic timestamp formats such as Mac Absolute Time and WebKit/Windows-style epoch references.

Explain why echo and echo -n can generate different hashes.

Identify big-endian and little-endian byte ordering and explain its evidence interpretation impact.

Prepare a structured forensic worksheet/report with commands, screenshots, results and explanations.

1.2 Tools Required

Tool	Purpose	Installation/Verification
bash terminal	Run shell commands and arithmetic tests	Available by default on Linux
python3	Validate conversions and timestamp calculations	python3 --version
bc	Base conversion through command line calculator	sudo apt install -y bc
xxd	Hex dump and ASCII view of data	sudo apt install -y xxd or vim-common
date	Unix timestamp conversion	Available by default on Linux
md5sum / sha256sum	Hash verification exercises	Available by default on Linux

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help

(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
# sudo apt install -y bc
Installing:
bc

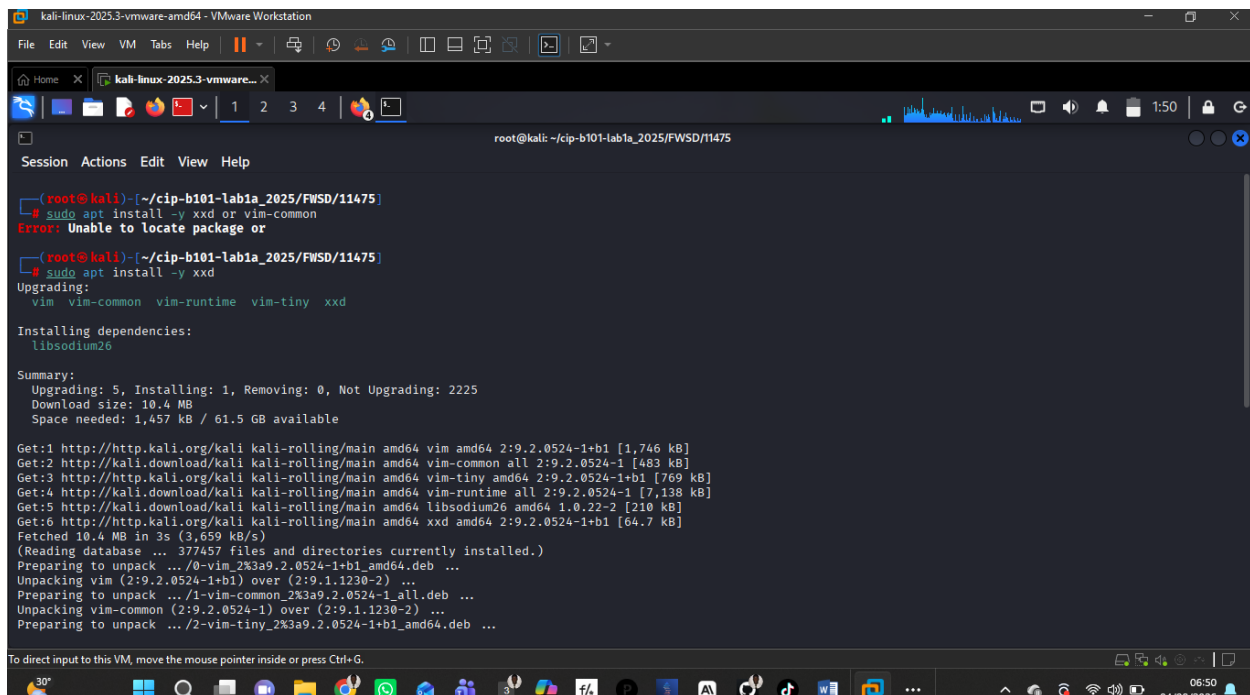
Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2230
  Download size: 103 kB
  Space needed: 242 kB / 61.5 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 bc amd64 1.07.1-4 [103 kB]
Fetched 103 kB in 2s (57.2 kB/s)
Selecting previously unselected package bc.
(Reading database ... 377439 files and directories currently installed.)
Preparing to unpack .../archives/bc_1.07.1-4_amd64.deb ...
Unpacking bc (1.07.1-4) ...
Setting up bc (1.07.1-4) ...
Processing triggers for kali-menu (2025.3.2) ...
Processing triggers for doc-base (0.11.2) ...
Processing 1 added doc-base file ...
Processing triggers for man-db (2.13.1-1) ...

(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
#

```

bc install



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
# sudo apt install -y xxd or vim-common
Error: Unable to locate package or

(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
# sudo apt install -y xxd
Upgrading:
vim vim-common vim-runtime vim-tiny xxd

Installing dependencies:
libsodium26

Summary:
Upgrading: 5, Installing: 1, Removing: 0, Not Upgrading: 2225
Download size: 10.4 MB
Space needed: 1,457 kB / 61.5 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 vim amd64 2:9.2.0524-1+b1 [1,746 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 vim-common all 2:9.2.0524-1 [483 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 vim-tiny amd64 2:9.2.0524-1+b1 [769 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 vim-runtime all 2:9.2.0524-1 [7,138 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 libsodium26 amd64 1.0.22-2 [210 kB]
Get:6 http://http.kali.org/kali kali-rolling/main amd64 xxd amd64 2:9.2.0524-1+b1 [64.7 kB]
Fetched 10.4 MB in 3s (3,659 kB/s)
(Reading database ... 377457 files and directories currently installed.)
Preparing to unpack .../0-vim_2%3a9.2.0524-1+b1_amd64.deb ...
Unpacking vim (2:9.2.0524-1+b1) over (2:9.1.1230-2) ...
Preparing to unpack .../1-vim-common_2%3a9.2.0524-1_all.deb ...
Unpacking vim-common (2:9.2.0524-1) over (2:9.1.1230-2) ...
Preparing to unpack .../2-vim-tiny_2%3a9.2.0524-1+b1_amd64.deb ...
```

Xxd installed

1.3 Student Working Folder

```
mkdir -p ~/cip-b101-lab1a
```

```
cd ~/cip-b101-lab1a
```

```
pwd
```

Screenshot requirement

Capture screenshots showing your terminal prompt and output for each main task. Your screenshots must clearly show your name or student ID in the working folder, file names, command used and result.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
root@kali)~)
# mkdir -p ~/cip-b101-lab1a_2025/FWSD/11475
root@kali)~)
# cd ~/cip-b101-lab1a_2025/FWSD/11475
root@kali)~/cip-b101-lab1a_2025/FWSD/11475)
# pwd
/root/cip-b101-lab1a_2025/FWSD/11475
root@kali)~/cip-b101-lab1a_2025/FWSD/11475)
#
```

A screenshot of my working folder in an absolute path form

2. Task A - Number Systems and Manual Conversion

This task builds your ability to interpret values written in base 10, base 2 and base 16. Forensic examiners encounter hexadecimal values in memory, disk sectors, file headers, hash outputs and metadata structures.

2.1 Quick Reference Table

Decimal	Binary	Hexadecimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9

Decimal	Binary	Hexadecimal
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

2.2 Practical Conversion Exercises

Convert 101010_2 to decimal manually. Show the weight of each bit from right to left.

Ans.

The process of the conversion here is multiplying each bit by its weight

$$(1 \times 32) + (0 \times 16) + (1 \times 8) + (0 \times 4) + (1 \times 2) + (0 \times 1)$$

$$32 + 0 + 8 + 0 + 2 + 0 = 42$$

$$101010_2 = 42_{10}$$

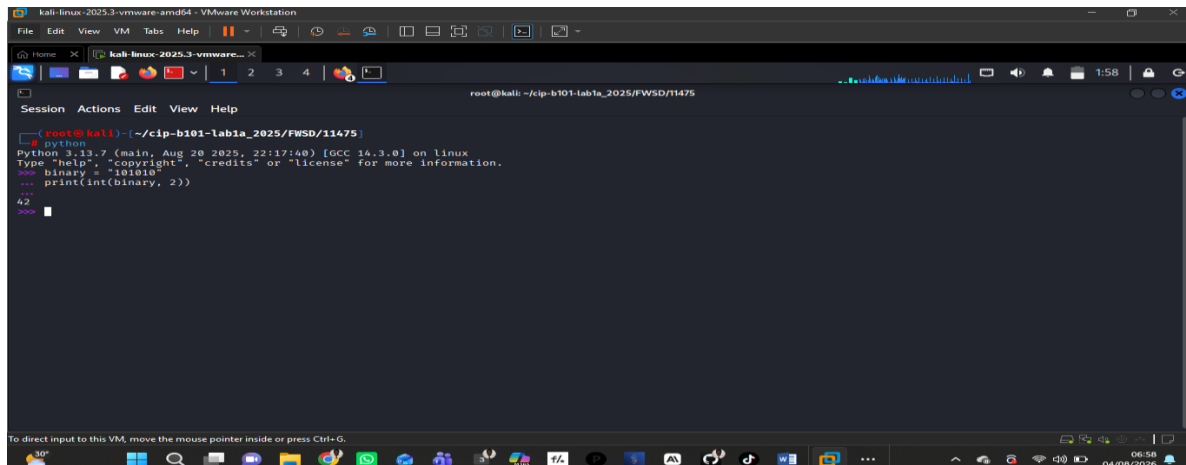
Linux Bash validation screenshot

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
root@kali: echo ${2#101010}
42
root@kali:

```

Python validation screenshot



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
python
Python 3.13.7 (main, Aug 20 2025, 22:17:40) [GCC 14.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> binary = "101010"
>>> print(int(binary, 2))
42
>>>
```

Convert $AB2_{16}$ to decimal manually. Remember that $A = 10$ and $B = 11$.

Ans.

Note: $A=10$ $B=11$

Calculating

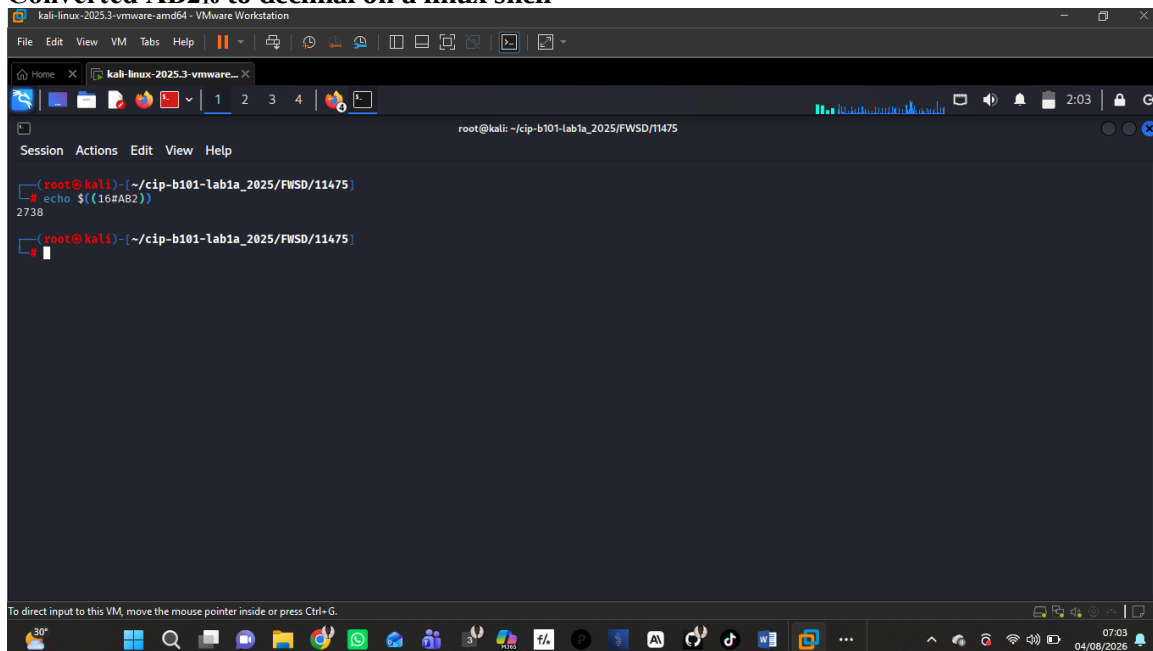
(value of a X weight value) + (value of b X its weight value) + (2 X 1)

$$2560 + 176 + 2$$

$$= 2738$$

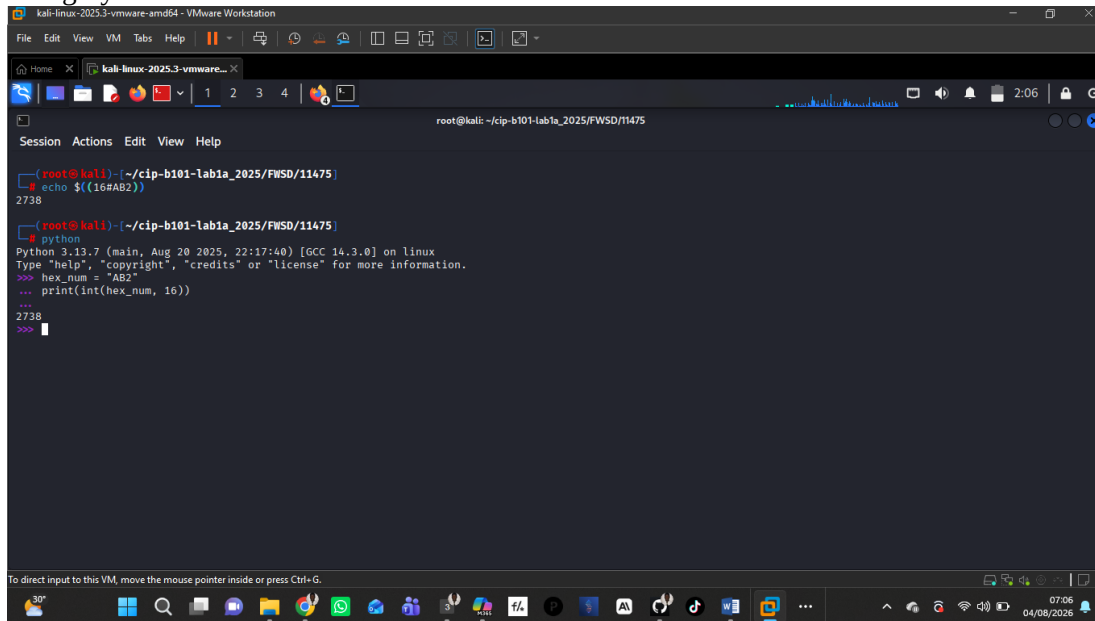
$$AB2_{16} = 2738_{10}$$

Converted $AB2_{16}$ to decimal on a linux shell



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
echo $((16#AB2))
2738
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
```

Using Python



```
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# echo $((16#AB2))
2738

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python
Python 3.13.7 (main, Aug 20 2025, 22:17:40) [GCC 14.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> hex_num = "AB2"
... print(int(hex_num, 16))
...
2738
>>>
```

Convert 126_{10} to binary manually by division by 2.

Ans.

Dividing 126 and every output by 2 in a table and recording the remainders

Division	Output	Remainders
$126 \div 2$	63	0
$63 \div 2$	31	1
$31 \div 2$	15	1
$15 \div 2$	7	1
$7 \div 2$	3	1
$3 \div 2$	1	1
$1 \div 2$	0	1

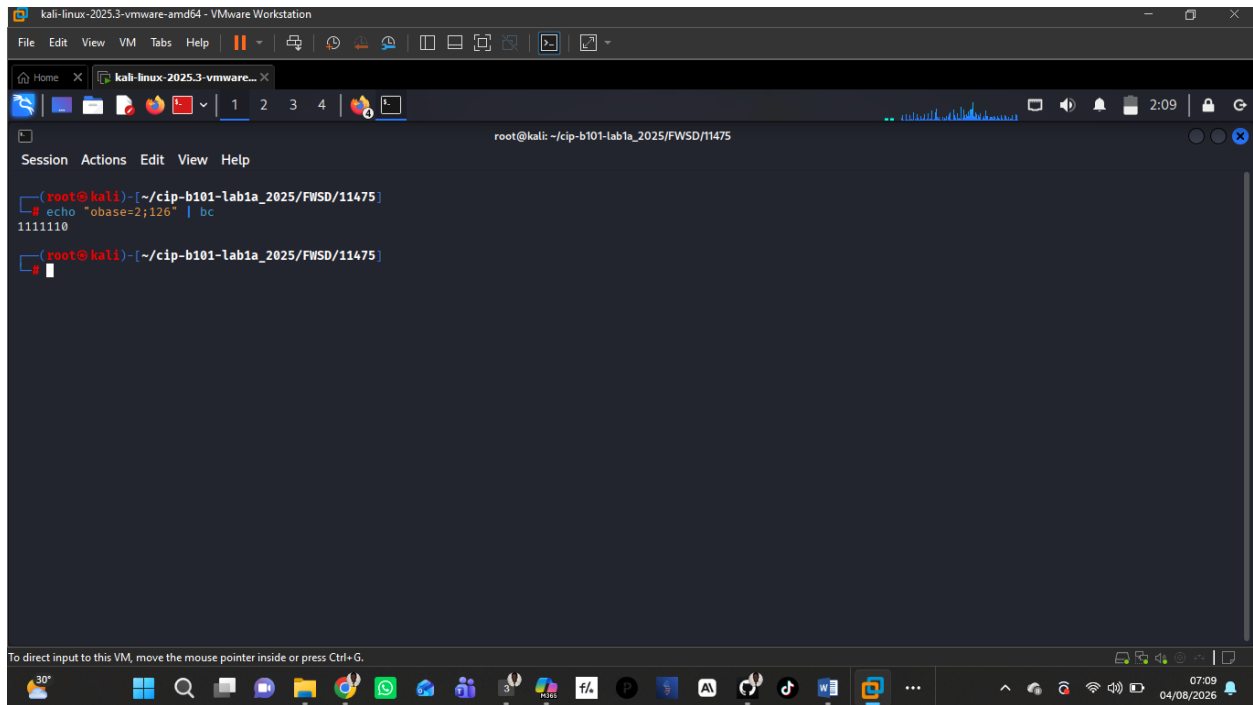
Write the remainders from the bottom

= 1111110_2

Therefore, $126_{10} = 1111110_2$

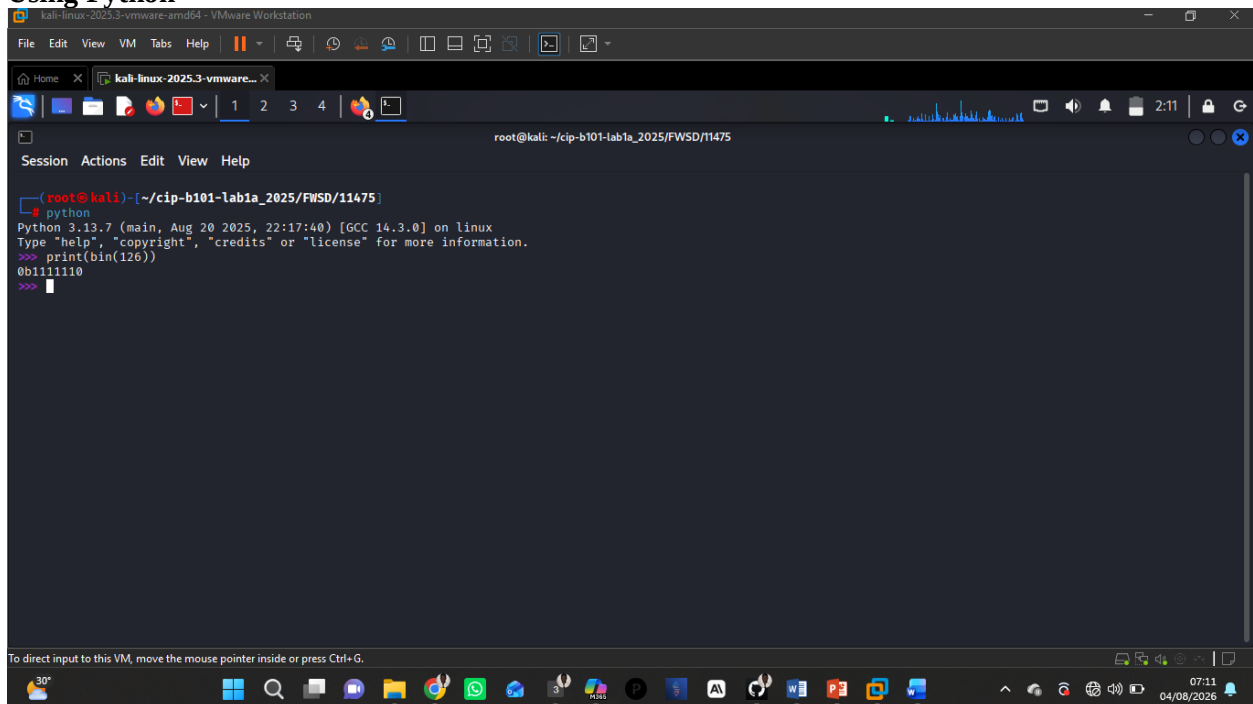
Linux shell

Using bc



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# echo "obase=2;126" | bc
1111110
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

Using Python



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python
Python 3.13.7 (main, Aug 20 2025, 22:17:40) [GCC 14.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print(bin(126))
0b1111110
>>>
```

Convert 2738_{10} to hexadecimal manually by division by 16.

Ans.

Note:

A: 10

B: 11

Divide 2738 and every result repeatedly by 16

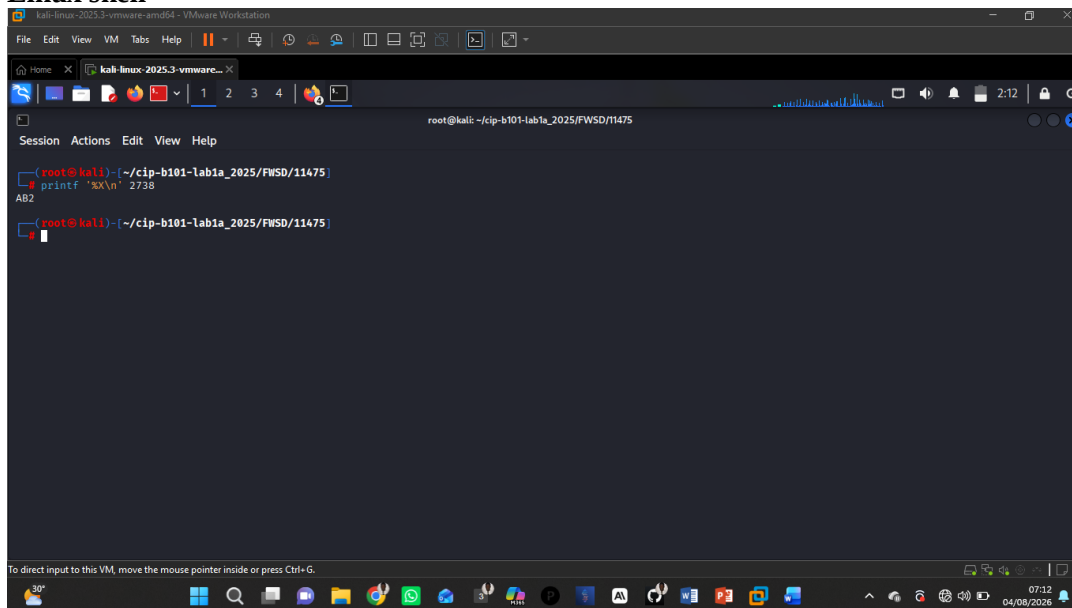
Division	Output	Remainders
$2738 \div 16$	171	2
$171 \div 16$	10	11
$10 \div 16$	0	10

Read remainders from bottom to top

$AB2_{16}$

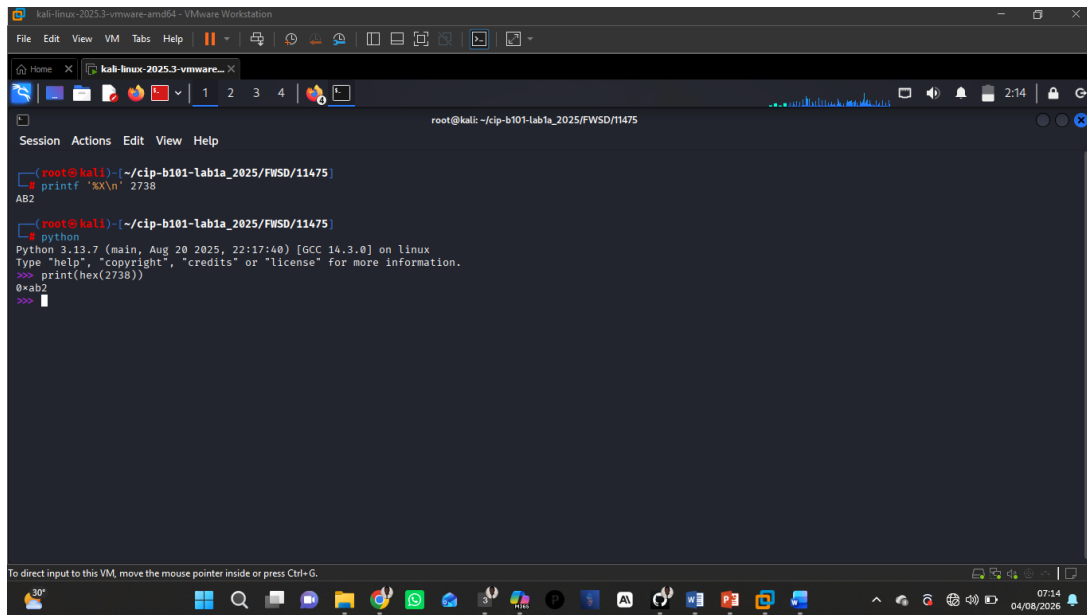
Therefore, $2738_{10} = AB2_{16}$

Linux shell



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
# printf "%X\n" 2738
AB2
(root@kali)~/cip-b101-lab1a_2025/FWSD/11475
#
```

Python



```
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# printf "%X\n" 2738
AB2

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python
Python 3.13.7 (main, Aug 20 2025, 22:17:40) [GCC 14.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print(hex(2738))
0xab2
>>>
```

Convert 101010110010_2 to hexadecimal by grouping bits in fours from the right.

Ans.

First I we will start with grouping of the bits into sets of four from the right

i.e 1010 1011 0010

count each group with their representatives from the reference table given

<u>Binary</u>	<u>Hex</u>
1010	A
1011	B
0010	2

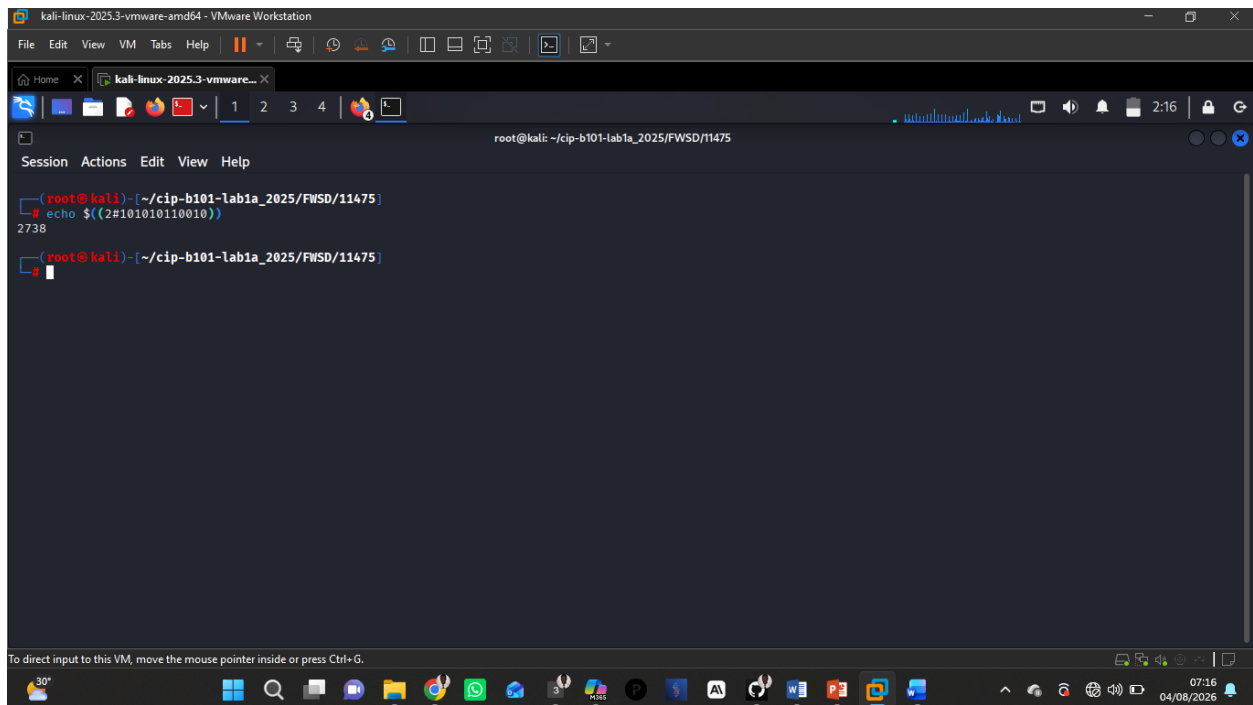
Therefore,

$$101010110010 = AB2_{16}$$

$$\text{So, } 101010110010 = AB2_{16}$$

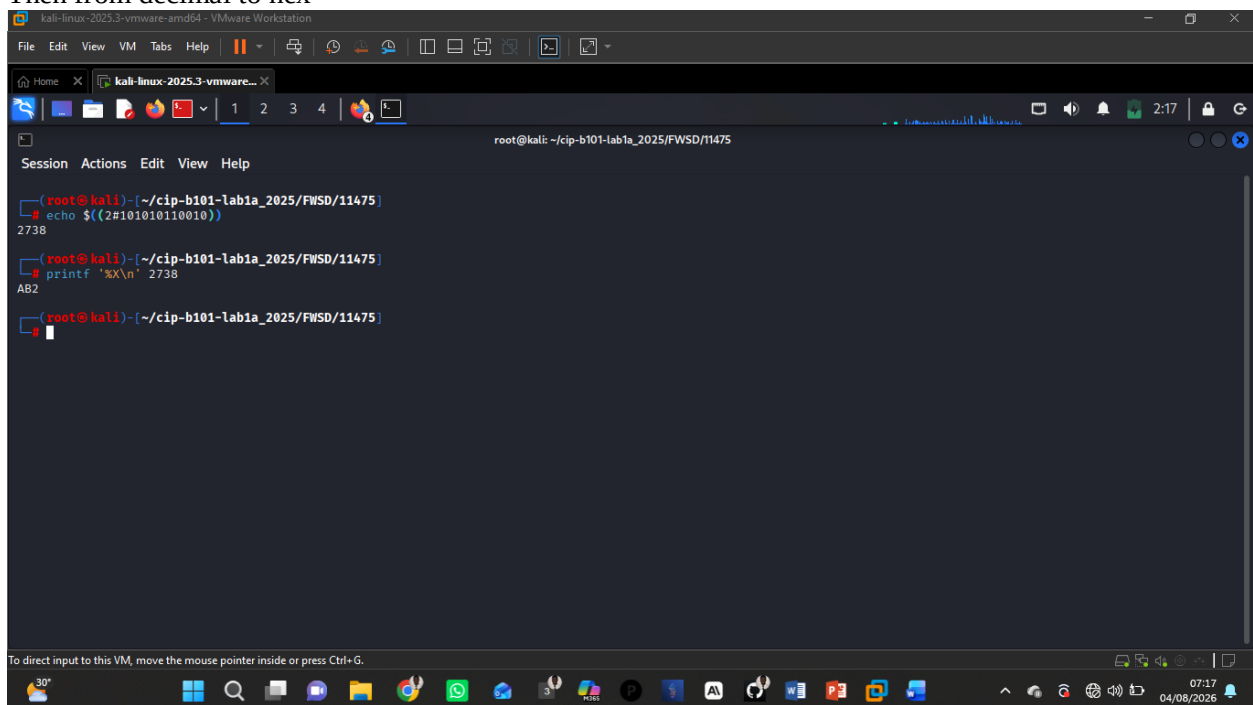
Linux shell

Converting binary vto decimal first:



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# echo $((2#101010110010))
2738
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

Then from decimal to hex



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# echo $((2#101010110010))
2738
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# printf '%x\n' 2738
AB2
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

2.3 Validate With Linux Shell and Python

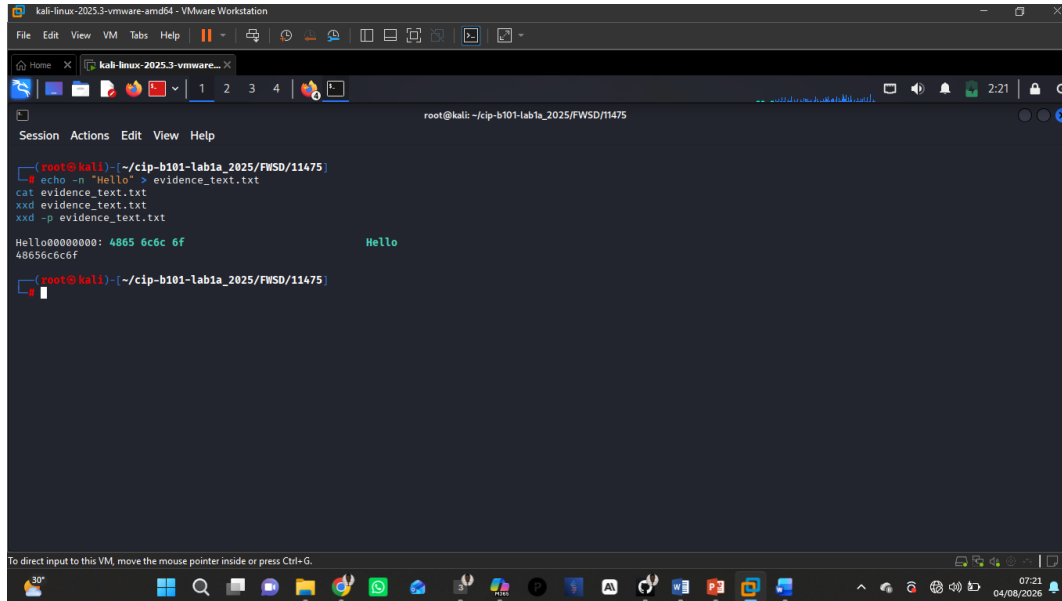
3. Task B - ASCII, UTF-8 and Hex Evidence Strings

Digital evidence is often stored as bytes. A byte can represent an ASCII character, part of a Unicode character, a number, or binary data. In this task, you will convert between text and hexadecimal representations.

3.1 Create Text Evidence and View as Hex

```
echo -n "Hello" > evidence_text.txt
cat evidence_text.txt
xxd evidence_text.txt
xxd -p evidence_text.txt
```

Ans.



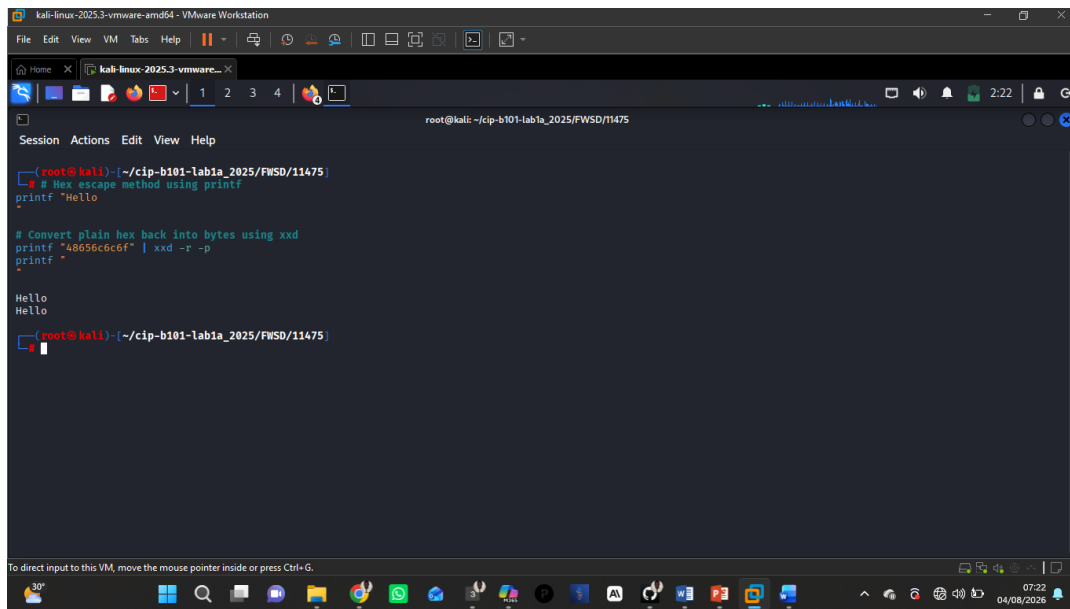
The screenshot shows a terminal window titled 'kali-linux-2025.3-vmware-amd64 - VMware Workstation'. The user is at the root prompt in a directory '~/.cip-b101-lab1a_2025/FWSD/11475'. The commands executed are: `echo -n "Hello" > evidence_text.txt`, `cat evidence_text.txt`, `xxd evidence_text.txt`, and `xxd -p evidence_text.txt`. The output of `xxd -p` is displayed as `Hello00000000: 4865 6c6c 6f` followed by the word 'Hello' on the next line. The terminal window has a dark background and a light-colored text. The VMware Workstation interface is visible around the terminal window.

3.2 Convert Hex to ASCII

```
# Hex escape method using printf
printf "Hello
"

# Convert plain hex back into bytes using xxd
printf "48656c6c6f" | xxd -r -p
printf "
"
```

Ans.



The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal is running a C program. The code includes a comment "# Hex escape method using printf", a printf statement "printf \"Hello\"", and another comment "# Convert plain hex back into bytes using xxd" followed by a printf statement "printf \"%8656c6f\" | xxd -r -p". The output of the program is "Hello" followed by "Hello" on a new line. The terminal prompt is "(root@kali)~/.cip-b101-lab1a_2025/FWSD/11475".

```
(root@kali)~/.cip-b101-lab1a_2025/FWSD/11475
# Hex escape method using printf
printf "Hello"

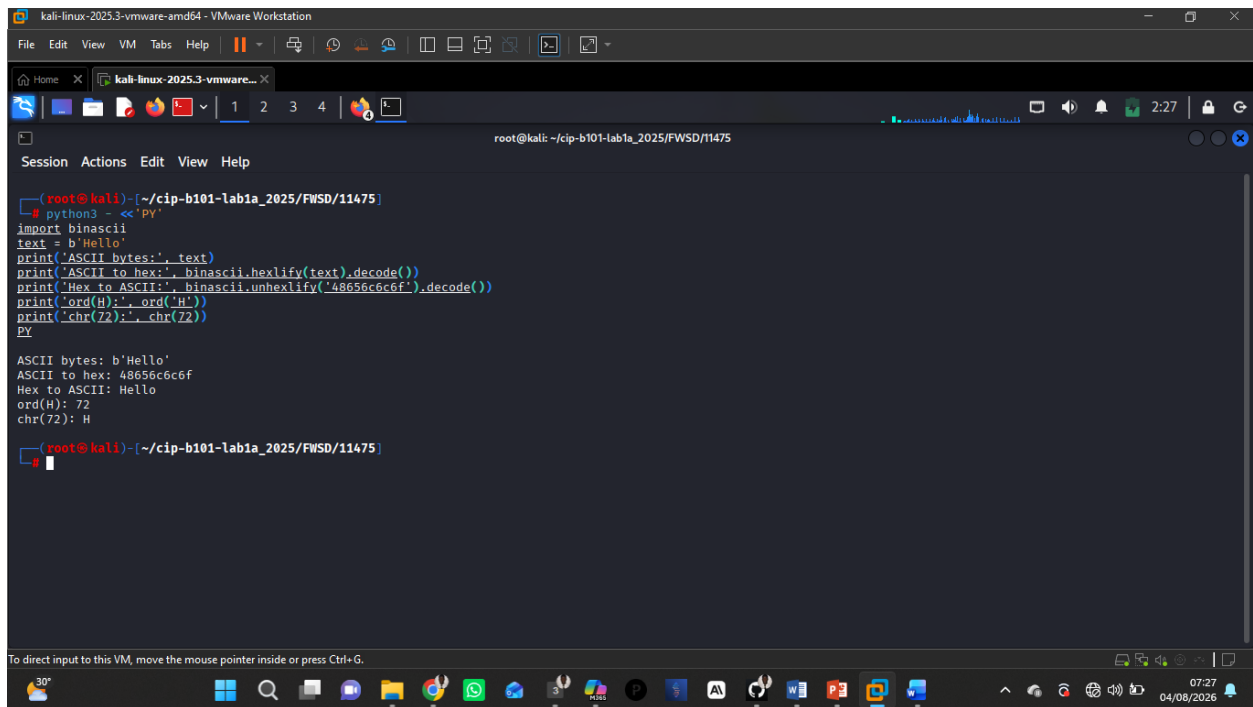
# Convert plain hex back into bytes using xxd
printf "%8656c6f" | xxd -r -p
printf "

Hello
Hello
(root@kali)~/.cip-b101-lab1a_2025/FWSD/11475
```

3.3 Python Conversion

```
python3 - <<'PY'
import binascii
text = b'Hello'
print('ASCII bytes:', text)
print('ASCII to hex:', binascii.hexlify(text).decode())
print('Hex to ASCII:', binascii.unhexlify('486556c6c6f').decode())
print('ord(H):', ord('H'))
print('chr(72):', chr(72))
PY
```

Ans.

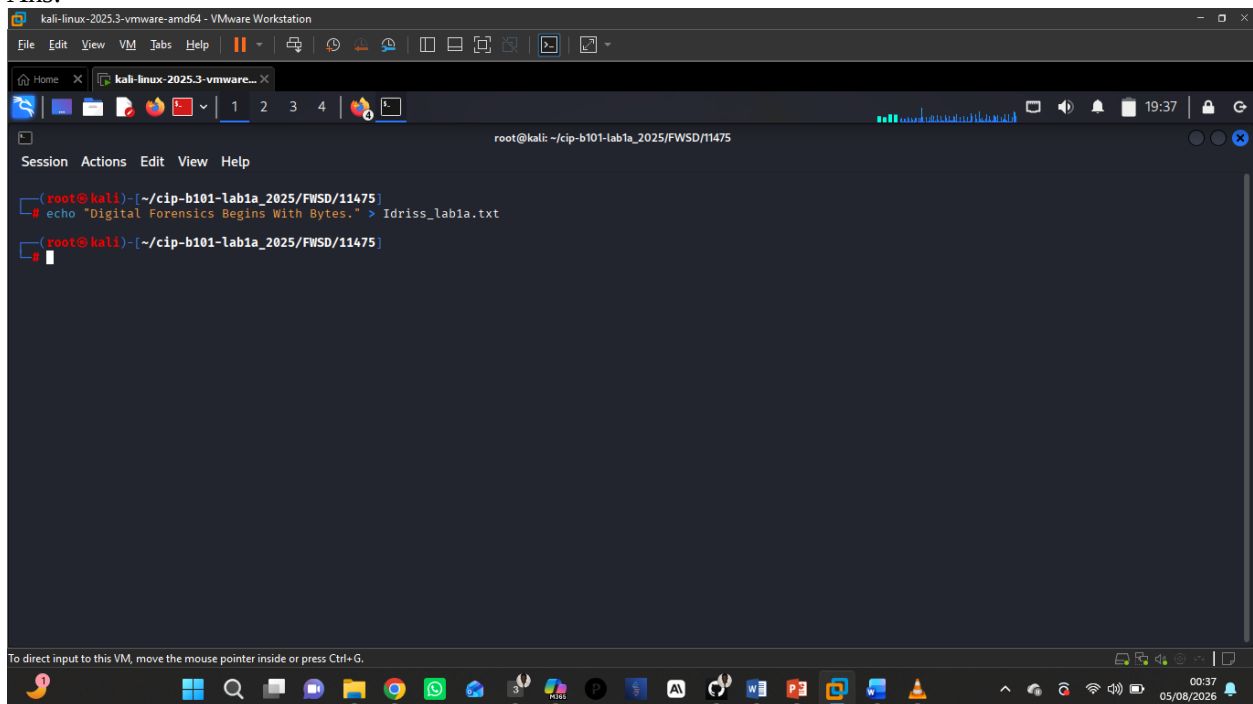


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python3 - <<'py'
import binascii
text = b'Hello'
print('ASCII bytes:', text)
print('ASCII to hex:', binascii.hexlify(text).decode())
print('Hex to ASCII:', binascii.unhexlify("48656c6c66").decode())
print('ord(H):', ord('H'))
print('chr(72):', chr(72))
py
ASCII bytes: b'Hello'
ASCII to hex: 48656c6c66
Hex to ASCII: Hello
ord(H): 72
chr(72): H
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

3.4 Student Exercise

Create a file named yourfirstname_lab1a.txt containing the text: Digital Forensics Begins With Bytes.

Ans.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# echo "Digital Forensics Begins With Bytes." > Idriss_lab1a.txt
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

Use xxd to display the file content in hexadecimal.

Ans.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

Session Actions Edit View Help

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# echo "Digital Forensics Begins With Bytes." > Idriss_lab1a.txt

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd Idriss_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963  Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279  s Begins With By
00000020: 7465 732e 0a                                     tes..

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
#
```

Extract the plain hex using xxd -p.

Ans.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

Session Actions Edit View Help

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# echo "Digital Forensics Begins With Bytes." > Idriss_lab1a.txt

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd Idriss_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963  Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279  s Begins With By
00000020: 7465 732e 0a                                     tes..

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd -p Idriss_lab1a.txt
4469676974616c20466f72656e7369637320426567696e73205769746820
42797465732e0a

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
#
```

Convert the plain hex back to readable text.

Ans.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

Session Actions Edit View Help

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# echo "Digital Forensics Begins With Bytes." > Idriss_lab1a.txt

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd Idriss_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963   Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279   s Begins With By
00000020: 7465 732e 0a                                tes..

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd -p Idriss_lab1a.txt
4469676974616c20466f72656e7369637320426567696e73205769746820
42797465732e0a

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# xxd -r -p <<< "4469676974616c20466f72656e7369637320426567696e7320576974682042797465732e0a"
Digital Forensics Begins With Bytes.

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
#
```

Explain why 0A may appear at the end of output when echo is used without -n.

Ans.

0A is the hexadecimal value for the newline character (\n) in ASCII. When you use echo without the -n option like example;

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

Session Actions Edit View Help

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
# echo "Digital Forensics Begins With Bytes."
Digital Forensics Begins With Bytes.

(root@kali)~/~/cip-b101-lab1a_2025/FWSD/11475
#
```

echo automatically appends a newline character to the end of the text, Therefore, the file actually contains: *Digital Forensics Begins With Bytes.*\n. And the newline appears in hexadecimal as 0a

Forensic explanation requirement

In your report, explain why an examiner should not assume every hexadecimal value is automatically readable text. Some bytes may represent binary structures, images, compressed data, or encrypted content.

Report:

A digital forensic examiner should not assume that every hexadecimal value represents readable text. Hexadecimal is simply a way of displaying raw binary data, and the meaning of a byte depends on how the data was created and stored. hexadecimal is also representation of raw data, not necessarily text. Some bytes represent characters, while others may represent program instructions, image content, compressed information, encrypted data, or other binary structures. A careful forensic examiner must determine the true nature of the data before interpreting it.

4. Task C - Epoch Time and Forensic Timestamps

Timestamps are central to forensic timelines. Incorrect conversion can place a suspect action at the wrong time. This task introduces Unix epoch time and compares it with other epoch systems mentioned in the course material.

4.1 Unix Epoch Conversion

```
# Current Unix timestamp
date +%s

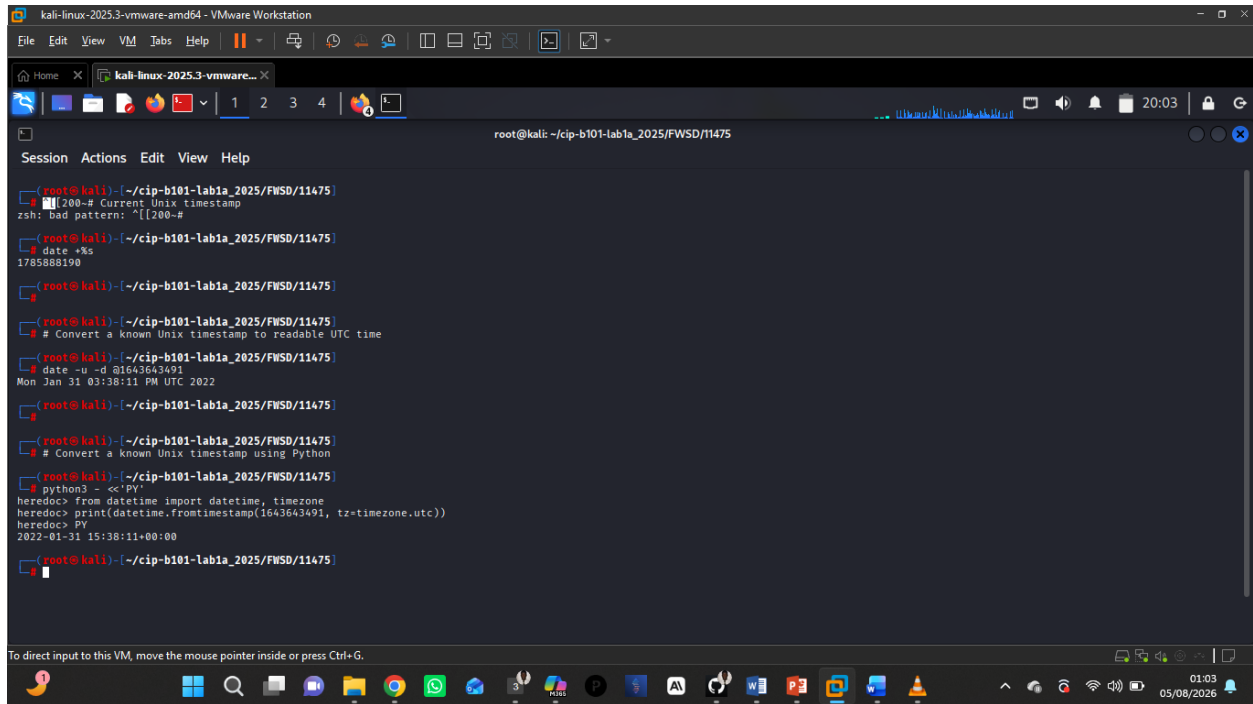
# Convert a known Unix timestamp to readable UTC time
date -u -d @1643643491

# Convert a known Unix timestamp using Python
python3 - <<'PY'
from datetime import datetime, timezone
```

```
print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
```

PY

Ans.



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# [[200-# Current Unix timestamp
zsh: bad pattern: "[[200-#

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date +%s
1785888190

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# # Convert a known Unix timestamp to readable UTC time

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date -u -d @1643643491
Mon Jan 31 03:38:11 PM UTC 2022

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# # Convert a known Unix timestamp using Python

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python3 - <<'PY'
heredoc> from datetime import datetime, timezone
heredoc> print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
heredoc> PY
2022-01-31 15:38:11+00:00

root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
```

4.2 Compare Local Time and UTC

```
date
```

```
date -u
```

```
python3 - <<'PY'
```

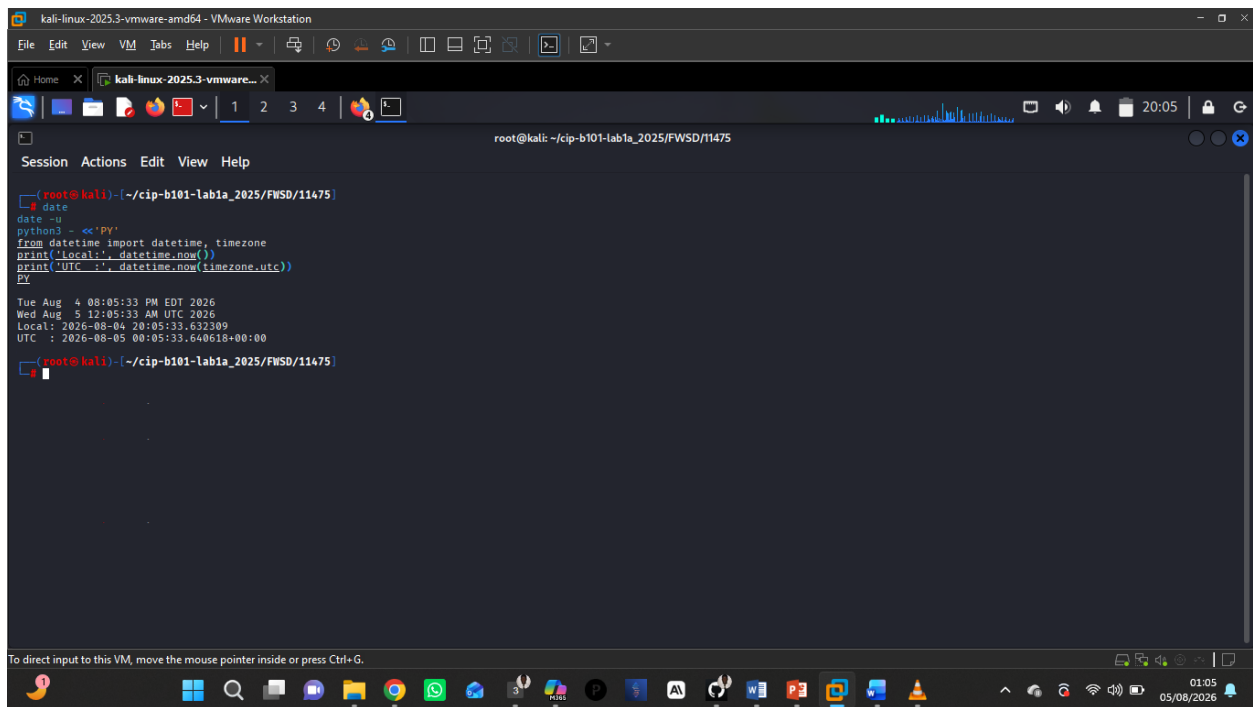
```
from datetime import datetime, timezone
```

```
print('Local:', datetime.now())
```

```
print('UTC :', datetime.now(timezone.utc))
```

PY

Ans.



The screenshot shows a Kali Linux terminal window titled 'kali-linux-2025.3-vmware-amd64 - VMware Workstation'. The terminal is running a Python script that prints the current date and time in local, UTC, and ISO 8601 formats. The output shows the current date as Tuesday, August 4, 2026, at 08:05:33 PM EDT. The terminal window has a dark theme and a menu bar with options like File, Edit, View, VM, Tabs, and Help. The bottom status bar shows the time as 01:05 on 05/08/2026.

```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date
date -u
python3 - <<'PY'
from datetime import datetime, timezone
print("local: ", datetime.now())
print("UTC : ", datetime.now(timezone.utc))
PY
Tue Aug 4 08:05:33 PM EDT 2026
Wed Aug 5 12:05:33 AM UTC 2026
Local: 2026-08-04 20:05:33.632309
UTC : 2026-08-05 00:05:33.640618+00:00

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
```

4.3 Mac Absolute Time / Cocoa Core Date Offset

Mac Absolute Time counts seconds since 2001-01-01 00:00:00 UTC. Unix time counts seconds since 1970-01-01 00:00:00 UTC. The difference between the two epochs is 978307200 seconds.

```
python3 - <<'PY'

from datetime import datetime, timezone

unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
print(int((mac_epoch - unix_epoch).total_seconds()))

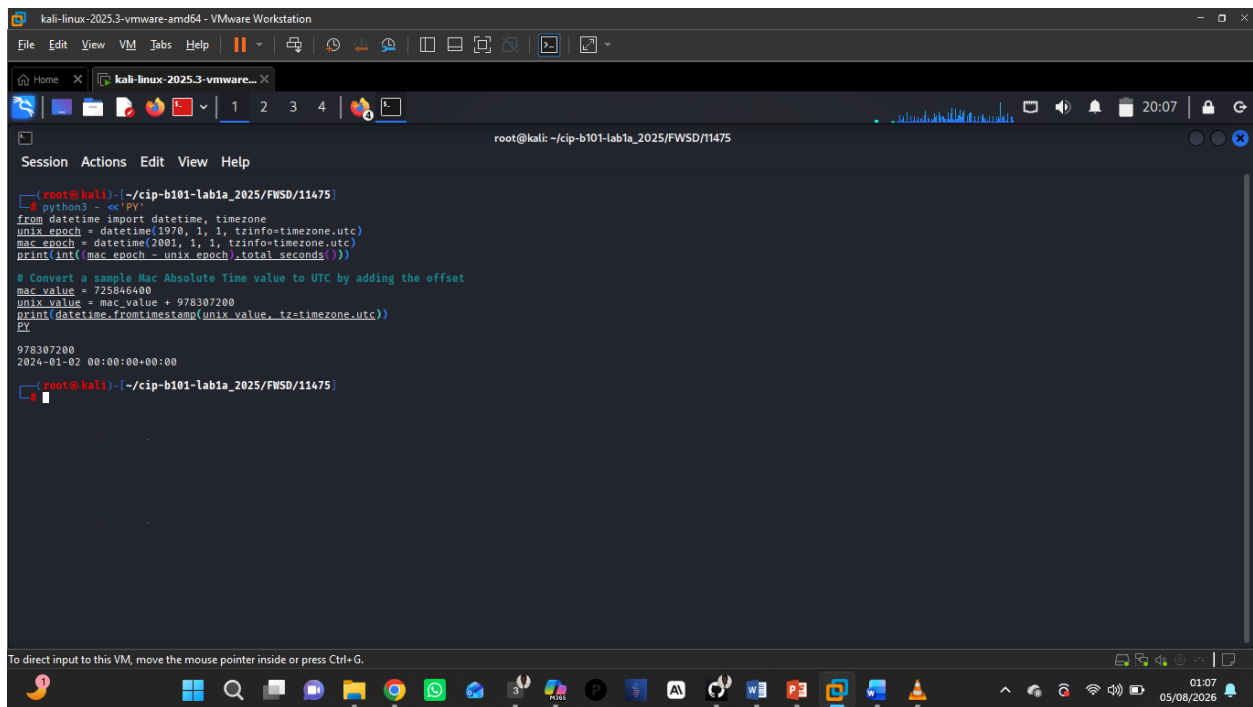
# Convert a sample Mac Absolute Time value to UTC by adding the offset
mac_value = 725846400

unix_value = mac_value + 978307200

print(datetime.fromtimestamp(unix_value, tz=timezone.utc))

PY
```

Ans.



4.4 Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds	date -u -d @1643643491	2022-01-31 15:38:11 UTC	
1675008832	Unix seconds	date -u -d @1675008832	2023-01-29 16:13:52 UTC	
725846400	Mac Absolute Time seconds	date -u -d "2001-01-01 UTC + 725846400 seconds	2024-01-02 00:00:00 UTC	
Current time	Unix seconds	date +%s	This varies depending on the time of execution	
Important timeline warning Always state whether your output is UTC or local time. In formal forensic work, timeline confusion caused by timezone mistakes can seriously weaken the reliability of findings.				

Screenshot references

```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date -u -d @1643643491
Mon Jan 31 03:38:11 PM UTC 2022

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date -u -d @1675008832
Sun Jan 29 04:13:52 PM UTC 2023

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# date -u -d "2001-01-01 UTC + 725846400 seconds"
Tue Jan 2 12:00:00 AM UTC 2024

(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

Timestamps are critical evidence in digital investigations because they help reconstruct the sequence of events. Examiners must always verify whether the displayed time is UTC or local time, since a timezone error can shift events by several hours and lead to incorrect conclusions about when user activity occurred. Unix timestamps count seconds from 1 January 1970 UTC, while Mac Absolute Time counts seconds from 1 January 2001 UTC.

5. Task D - Hashing and the Newline Problem

Hashing is used to verify that evidence has not changed. Small changes, including one extra newline character, produce a different hash.

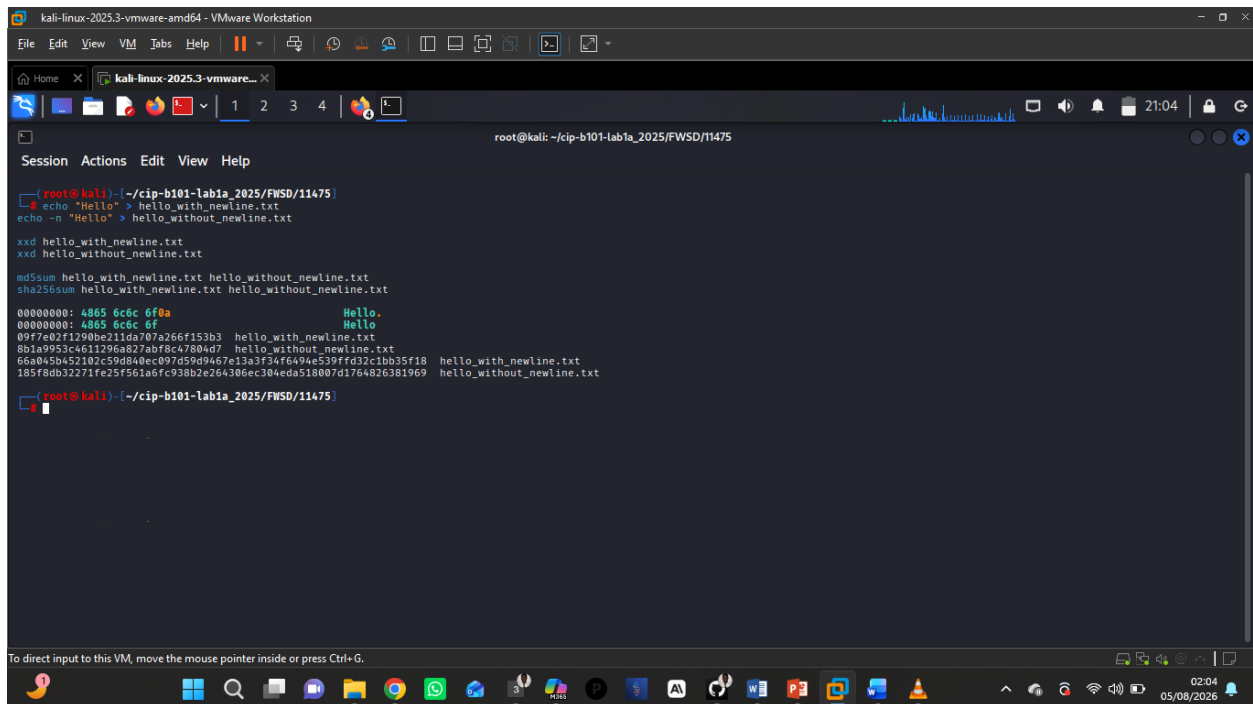
5.1 Create Two Similar Files

```
echo "Hello" > hello_with_newline.txt
echo -n "Hello" > hello_without_newline.txt

xxd hello_with_newline.txt
xxd hello_without_newline.txt

md5sum hello_with_newline.txt hello_without_newline.txt
sha256sum hello_with_newline.txt hello_without_newline.txt
```

Ans.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
echo "Hello" > hello_with_newline.txt
echo -n "Hello" > hello_without_newline.txt
xxd hello_with_newline.txt
xxd hello_without_newline.txt
md5sum hello_with_newline.txt hello_without_newline.txt
sha256sum hello_with_newline.txt hello_without_newline.txt
00000000: 4865 6c6c 6f0a                Hello.
00000000: 4865 6c6c 6f                Hello
09f7e02f1290be211da787a266f153b3  hello_with_newline.txt
801a9953c4611296a827abf0c47804d7  hello_without_newline.txt
66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffrd32c1bb35f18  hello_with_newline.txt
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969  hello_without_newline.txt
root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
```

5.2 Explain the Difference

File 1 contains the characters H e l l o plus a newline byte.

File 2 contains only H e l l o without the newline byte.

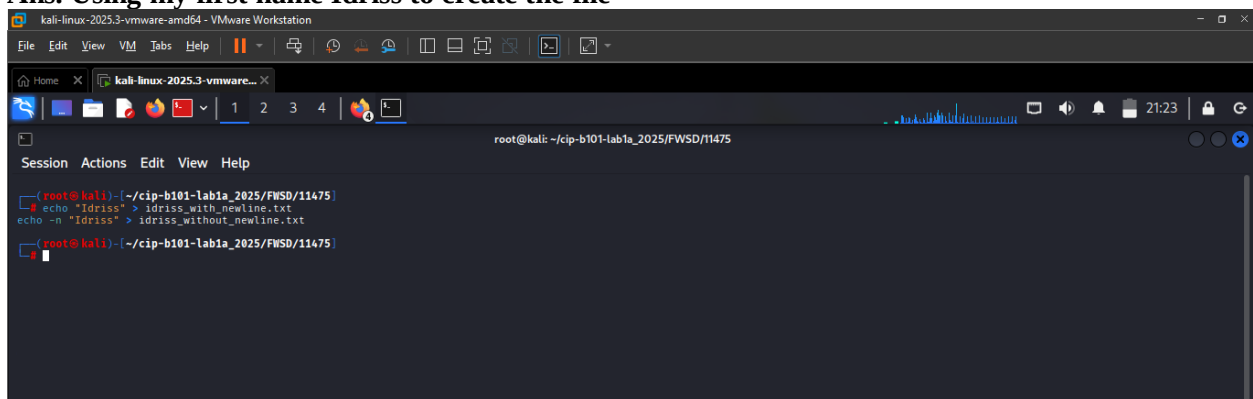
The visible text may look the same, but the underlying bytes are different.

Because cryptographic hashes are byte-sensitive, the MD5 and SHA-256 values will be different.

5.3 Student Exercise

Create two files containing your first name: one using echo and one using echo -n.'

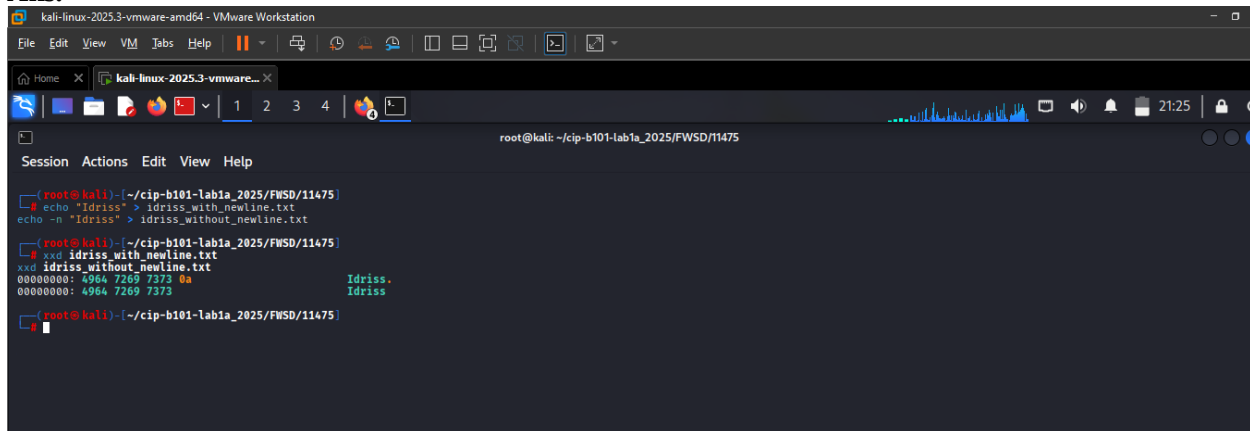
Ans. Using my first name Idriss to create the file



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
echo "Idriiss" > idriiss_with_newline.txt
echo -n "Idriiss" > idriiss_without_newline.txt
root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
```

Use xxd to compare the byte-level difference.

Ans.



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
# echo "Idriiss" > idriiss_with_newline.txt
echo -n "Idriiss" > idriiss_without_newline.txt

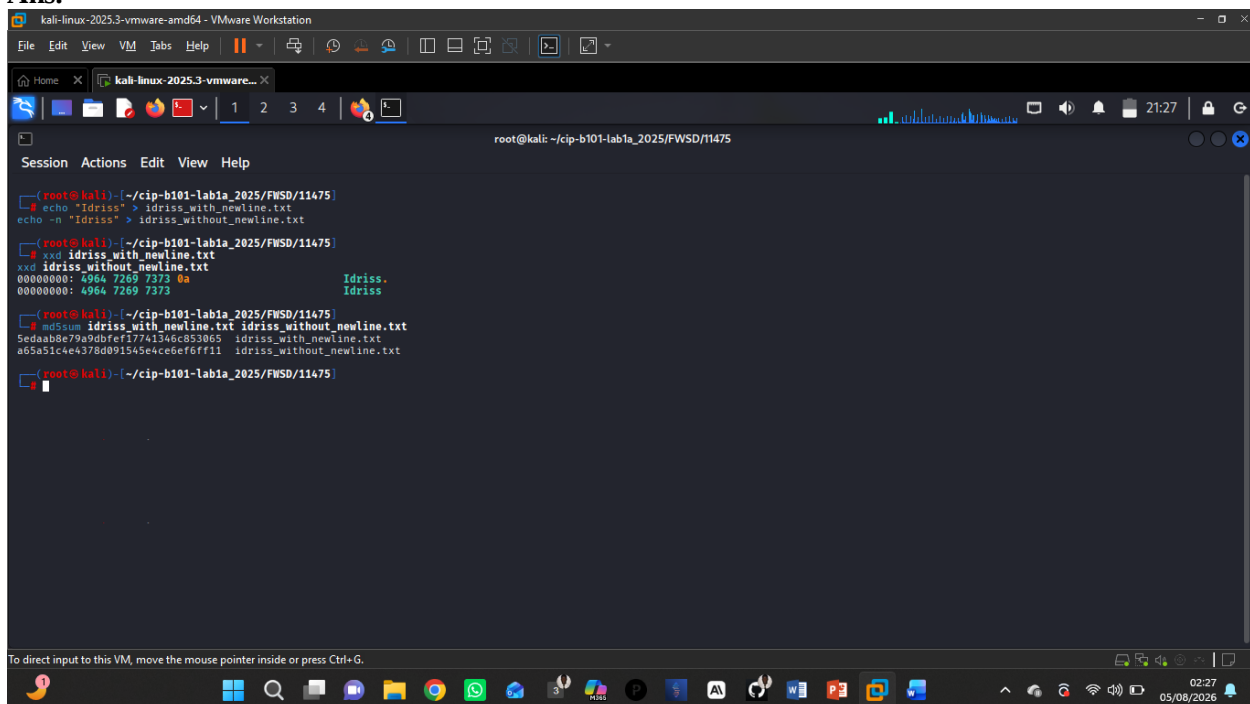
root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
# xxd idriiss_with_newline.txt
xxd idriiss_without_newline.txt
00000000: 4964 7269 7373 0a          Idriiss.
00000000: 4964 7269 7373              Idriiss

root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
```

The only difference is the additional byte

Calculate MD5 and SHA-256 for both files.

Ans.



```
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
# echo "Idriiss" > idriiss_with_newline.txt
echo -n "Idriiss" > idriiss_without_newline.txt

root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
# xxd idriiss_with_newline.txt
xxd idriiss_without_newline.txt
00000000: 4964 7269 7373 0a          Idriiss.
00000000: 4964 7269 7373              Idriiss

root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
# md5sum idriiss_with_newline.txt idriiss_without_newline.txt
5edaab8e79a9dbef17741346c853065 idriiss_with_newline.txt
a65a51c4e4378d091545e4cebe6f6ff11 idriiss_without_newline.txt

root@kali) [~/cip-b101-lab1a_2025/FWSD/11475]
```

Explain why the hash values differ.

Explanation:

Hash functions such as MD5 and SHA-256 calculate a digital identity based on every byte contained in a file. Although both files appear to contain the same visible text, their contents are not identical.

Because the first file includes an extra newline byte (0A), the data being hashed is different. Cryptographic hash algorithms are extremely sensitive to changes, so even a single-byte difference produces a completely different hash value.

State which commands you would use if you needed to hash exactly the visible characters only.

Ans.

```
echo -n "Idriss" > idriss_without_newline.txt
```

The echo command creates a file containing the name plus a newline character (0A), while echo -n creates a file containing only the visible characters. The extra newline byte changes the file's binary content, causing completely different MD5 and SHA-256 hash values

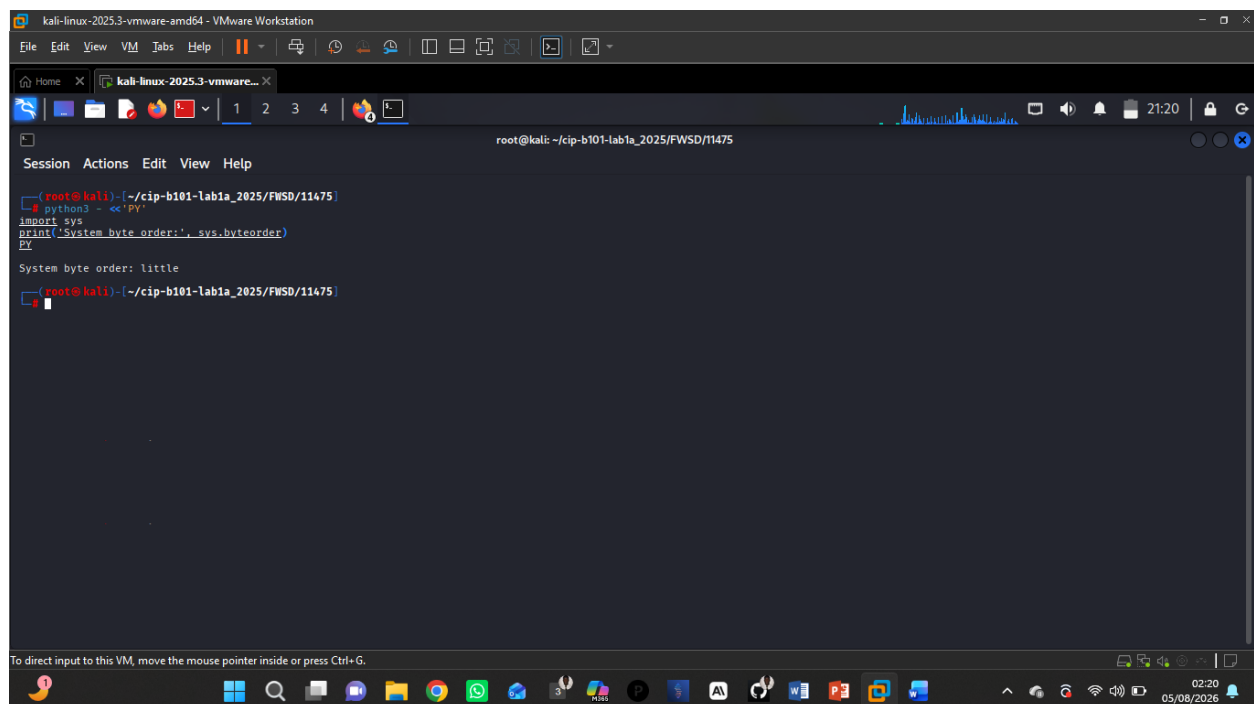
6. Task E - Endianness and Data Stored in Memory

Endianness describes the order in which bytes are stored in memory. Digital forensics analysts must understand this when interpreting raw memory, file structures, disk metadata and binary fields.

6.1 Check Byte Order

```
python3 - <<'PY'
import sys
print('System byte order:', sys.byteorder)
PY
```

Ans.

A screenshot of a Kali Linux terminal window running within a VMware Workstation. The terminal shows a Python script being executed. The script imports the sys module and prints the system byte order. The output is 'System byte order: little'. The terminal window has a dark background with light-colored text. The VMware interface is visible at the top and bottom of the terminal window.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
# python3 - <<'PY'
import sys
print('System byte order:', sys.byteorder)
PY
System byte order: little
(root@kali) ~/cip-b101-lab1a_2025/FWSD/11475
#
```

6.2 Interpret the Same Bytes in Two Ways

```
python3 - <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
PY
```

Ans.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
Session Actions Edit View Help
python3 - <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
PY
Raw bytes: 78563412
Interpreted as little-endian: 305419896
Interpreted as big-endian : 2018915346
root@kali: ~/cip-b101-lab1a_2025/FWSD/11475
```

6.3 Student Exercise

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16,777,216	echo 01000000 \ xxd -r -p \ od - An -t u4	Same bytes interpreted differently
FF000000	255	4,278,190,080	echo FF000000 \ xxd -r -p \ od - An -t u4	FF is least significant in little-endian but most significant in big-endian.

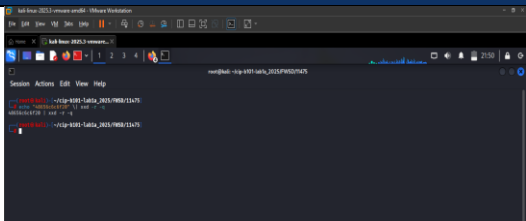
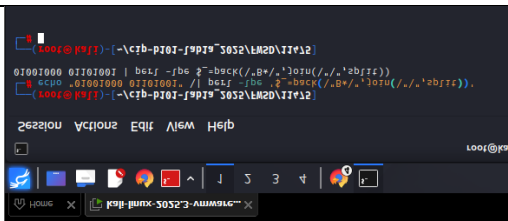
Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
00000100	65,536	256	<code>echo 00000100 \\ xxd -r -p \\\ od - An -t u4</code>	Byte ordering completely changes the value.
78563412	305,419,896	2,018,915,346	<code>echo 78563412 \\ xxd -r -p \\\ od - An -t x4</code>	Common example used to demonstrate endianness.
Forensic explanation requirement In your own words, explain how the same four bytes can produce different decimal values depending on byte order. Relate this to interpreting disk metadata or memory evidence.				

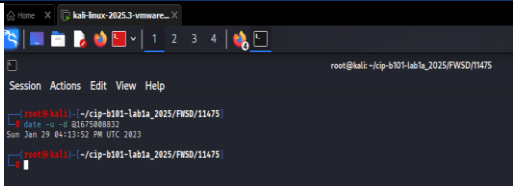
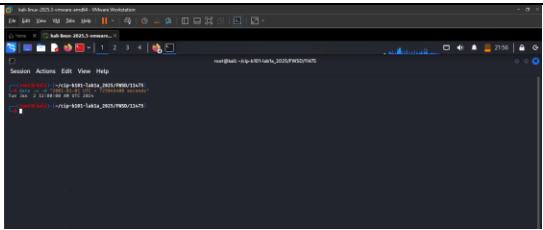
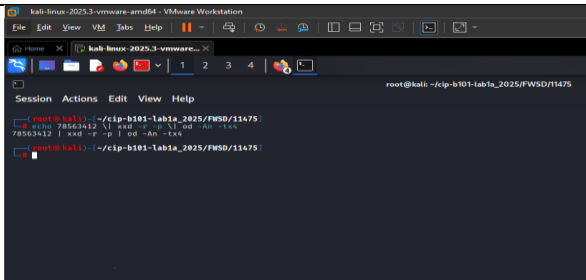
Ans.

To convert the four-byte hexadecimal values, interpret the bytes in both little-endian and big-endian order. A sequence of four bytes can represent completely different decimal values depending on the byte order used by the system. In big-endian format, the most significant byte is stored first, while in little-endian format, the least significant byte is stored first.

7. Task F - Mini Forensic Interpretation Case

You have recovered the following values from a training evidence source. Decode and interpret them using the techniques practiced above.

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	Hexadecimal data. Decoding the valid hex bytes 48 65 6c 6c 6f 20 gives "Hello " in ASCII. As understand	
EVID-002	01001000 01101001	Binary values converted to ASCII. 01001000 = 72 decimal = H and 01101001 = 105 decimal	

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-003	1675008832	Unix Epoch timestamp. Converts to 2023-01-29 16:13:52 UTC. This represents the date and time an event occurred.	
EVID-004	725846400 (Mac Absolute Time)	Mac Absolute Time begins on 1 January 2001. Conversion gives 2024-01-02 00:00:00 UTC.	
EVID-005	78563412 as little-endian and big-endian	As Big-Endian: 0x78563412 = 2,018,915,346. As Little-Endian: reversed to 0x12345678 = 305,419,896. The same bytes produce different values depending on byte order As i explained above	

7.1 Required Case Narrative

State what each evidence item represents after decoding.

For timestamp values, state the readable UTC time and the method used.

For byte-order values, state both interpretations and why they differ.

Explain which item would be useful for timeline analysis and which item would be useful for text/content interpretation.

Ans.

The first column contained hexadecimal data that translated to readable ASCII text, demonstrating how hexadecimal values can represent human-readable information when interpreted using the correct character encoding. The binary evidence item was decoded into the text "Hi", showing how binary values ultimately map to ASCII characters.

The third column was identified as a Unix Epoch timestamp. After conversion, it indicated an event occurring on 29 January 2023 at 16:13:52 UTC. The fourth column was recognized as a Mac Absolute Time value, which uses a different epoch beginning on 1 January 2001. Conversion revealed a date of 2 January 2024 at 00:00:00 UTC..

8. Submission Requirements

Required output file name

Submit your report as: CIP-B101_Lab1A_FirstName_LastName.pdf

8.1 Your Report Must Contain

Cover page with course code, lab title, full name, student ID, date and platform used.

Completed conversion worksheet for decimal, binary and hexadecimal tasks.

Screenshots of shell/Python validation commands.

ASCII/hex conversion evidence and explanation.

Timestamp conversion table with UTC result clearly stated.

Hash comparison table for files with and without newline characters.

Endianness worksheet and explanation.

Mini forensic case interpretation narrative.

Reflection: one paragraph on why number systems and timestamps matter in digital forensics.

Reflection:

This lab provided practical experience in handling digital evidence at the byte level. The exercises demonstrated how data can be represented in different formats such as hexadecimal, binary, timestamps, and byte ordering. The activities emphasized the importance of accurate data interpretation during digital forensic investigations. the exercises highlighted a key principle of digital forensics: examiners must analyses evidence based on its actual binary content rather than its visible appearance. Small differences in file contents, timestamps, or byte order can significantly affect investigative findings.

8.2 Minimum Screenshot Checklist

No.	Screenshot Evidence	Included?
1	Working folder creation and pwd output	Yes / No
2	Binary/hex/decimal conversion commands	Yes / No
3	xxd output for text evidence	Yes / No
4	Hex-to-ASCII conversion output	Yes / No
5	Unix timestamp conversion output	Yes / No
6	Mac Absolute Time conversion output	Yes / No

No.	Screenshot Evidence	Included?
7	Hash comparison using md5sum and sha256sum	Yes / No
8	Endianness Python output	Yes / No
9	Mini-case evidence interpretation commands	Yes / No

9. Marking Rubric

Assessment Area	Marks	What the Instructor Should Check
Manual number-system conversions	15	Correct method, weights/remainers shown, accurate final results.
Command-line and Python validation	15	Commands are correct, outputs are captured and interpreted.
ASCII/hex/UTF-8 understanding	15	Student can explain byte-level text representation and newline issue.
Timestamp conversion and timezone awareness	20	UTC/local distinction is clear; Unix and Mac time are converted correctly.
Hashing and evidence integrity explanation	15	Student explains why byte differences change hashes.
Endianness interpretation	10	Little-endian and big-endian differences are correctly explained.
Report quality and screenshots	10	Report is organized, complete and evidence screenshots are clear.
Pass standard A trainee should be considered competent if they can explain the meaning of the outputs, not merely run the commands.		

10. Reference Links from the Course Material

https://www.gnu.org/software/bash/manual/html_node/Shell-Arithmetic.html

<https://linuxize.com/post/bash-printf-command/>

<https://en.wikipedia.org/wiki/Unicode>

https://en.wikipedia.org/wiki/Coordinated_Universal_Time

<https://stackoverflow.com/questions/5693360/why-is-an-md5-hash-created-by-python-different-from-one-created-using-echo-and-m>

<https://www.iar.com/knowledge/learn/programming/migration-techniques-for-different-endianness/>